SPHOORTHY ENGINEERING COLLEGE
(UGC AUTONOMOUS)

A MAJOR PROJECT REPORT ON

KG-RAG: An Ontology-Driven Knowledge Graph Framework for Intelligent Campus Information Retrieval and Natural Language Question Answering

Submitted in partial fulfillment of the academic requirements for the award of the degree of
BACHELOR OF TECHNOLOGY
in
COMPUTER SCIENCE AND ENGINEERING

By Student Research Team:
LUCKY (22JC1A05XX) · SU-6372 Project Cohort

Under the Esteemed Guidance of:
Department Faculty Guide & Head of the Department
Department of Computer Science and Engineering

SPHOORTHY ENGINEERING COLLEGE
(An UGC Autonomous Institution · Accredited by NAAC with 'A' Grade)
Approved by AICTE, New Delhi & Affiliated to JNTU Hyderabad
Sagar Road, Nadargul, Hyderabad, Telangana 501510

Academic Year: 2025–2026

SPHOORTHY ENGINEERING COLLEGE

Autonomous Institution | Accredited by NAAC Grade 'A' | Affiliated to JNTUH
Department of Computer Science and Engineering

CERTIFICATE OF BONAFIDE WORK

This is to certify that the major project report entitled **'KG-RAG: An Ontology-Driven Knowledge Graph Framework for Intelligent Campus Information Retrieval and Natural Language Question Answering in Higher Education Institutions'** is a bonafide work carried out by the student research team of B.Tech IV Year in partial fulfillment for the award of the Degree of **Bachelor of Technology in Computer Science and Engineering** from Sphoorthy Engineering College, Hyderabad during the academic year 2025–2026.

The results embodied in this major project report have not been submitted to any other University or Institute for the award of any degree or diploma. The software algorithms, Knowledge Graph architectures, and testing pipelines documented herein were developed and validated under our supervision.

Internal Guide	**Head of Department**	**External Examiner**
Assistant Professor	Prof. Kiran B. M.	JNTUH Appointed
Dept. of CSE	Dept. of CSE	Viva-Voce Board

CANDIDATE DECLARATION

We hereby declare that the work presented in this major project report entitled **'KG-RAG: An Ontology-Driven Knowledge Graph Framework for Intelligent Campus Information Retrieval and Natural Language Question Answering'** submitted to the Department of Computer Science and Engineering, Sphoorthy Engineering College, Hyderabad, is our original contribution. We have adhered to all academic integrity principles and acknowledged all external literature, datasets, and open-source computational libraries appropriately.

ACKNOWLEDGEMENTS

We express our deep gratitude and heartfelt thanks to our respected Management, Principal, and **Prof. Kiran B. M.**, Head of the Department of Computer Science & Engineering, for providing the departmental infrastructure, computational resources, and guidance throughout the project development lifecycle.

We are immensely indebted to our Project Guide for continuous constructive critiques, algorithmic insights, and technical direction in formalizing the institutional ontology and designing the NetworkX traversal pipeline.

Finally, we thank our faculty members, academic colleagues, and families for their unwavering encouragement during this research.

EXECUTIVE ABSTRACT

Higher education institutions manage complex, multi-relational, and dynamic information ecosystems encompassing departments, degrees, course prerequisites, faculty qualifications, fee structures, physical facilities, placement histories, and circular notices. Conventional information access approaches—static websites and keyword-matching search bars—fail to resolve multi-hop, relational queries such as connecting a course code to the department offering it, the assigned professor, and the designated laboratory room. While modern Large Language Models (LLMs) can generate natural language, deploying them directly for institutional question answering introduces severe limitations: high API costs, significant network latencies (>1,500 ms), privacy concerns, and hallucinated factual answers.

To address these critical shortcomings, this dissertation presents **KG-RAG**, an intelligent, ontology-driven campus knowledge graph question answering and administrative management system developed for **Sphoorthy Engineering College**. The core architecture constructs a formal property directed graph (NetworkX) consisting of **422 nodes across 22 entity types** and **736 typed edges across 27 relationship types**. The Natural Language Processing pipeline couples priority keyword rules with TF-IDF cosine similarity ((1,2) n-grams) and regex-based entity extraction with fuzzy matching (difflib, 0.8 cutoff), achieving **95.0%–100.0% intent classification accuracy** in sub-millisecond execution times (0.547 ms) on ordinary CPU hardware.

Beyond retrieval, KG-RAG integrates session-based dialogue state tracking for multi-turn conversational follow-up, an interactive D3.js force-directed graph visualizer, and three dedicated web portals: a **Student Study Hub** featuring in-browser PDF lecture notes reading and instant 'Ask Bot' knowledge linking; a **Faculty Dashboard** featuring live ontology-backed teaching allocations (Faculty -[teaches]-> Subject and Subject -[uses_lab]-> Lab) with an interactive profile switcher; and an **Admin Console** capable of dynamic hot-reloading that rebuilds the in-memory graph in **7.78 ms** without server restarts.

Comprehensive empirical benchmarking across 40 evaluation queries confirms an average end-to-end HTTP response latency of **7.81 ms** (P95: 10.44 ms), a system throughput of **1,828.6 queries per second**, and a minimal memory footprint of **0.36 MB**. The system completely passes all 62 automated ontology and ER validation tests, providing a factual, zero-cost, and robust institutional assistant ready for enterprise campus deployment.

Keywords: Knowledge Graph Question Answering (KGQA), Retrieval-Augmented Generation (RAG), Domain Ontology, NetworkX Directed Graph, TF-IDF Cosine Similarity, Dialogue State Tracking, Sphoorthy Engineering College.

TABLE OF CONTENTS

CHAPTER / SECTION	TITLE / DESCRIPTION	PAGE NO.
—	Preliminary Pages (Certificate, Declaration, Acknowledgements, Abstract)	i – iv
CHAPTER 1	**INTRODUCTION & INSTITUTIONAL CONTEXT** Overview, Sphoorthy Engg. College Profile, The Campus Information Bottleneck, Key Contributions	7
CHAPTER 2	**BACKGROUND, MOTIVATION & PROBLEM FORMULATION** Relational Campus Architecture, Search Paradigms Comparison, Formal Graph & NLP Formulation	9
CHAPTER 3	**LITERATURE REVIEW & BASE PAPER ANALYSIS** Foundational Paradigms, Critical Review of 12 Base Papers, Feature Matrix, Architectural Gap Analysis	11
CHAPTER 4	**PROJECT OBJECTIVES & TECHNICAL SCOPE** Primary Technical Objectives, Functional Deliverables, Institutional Constraints	14
CHAPTER 5	**SYSTEM ARCHITECTURE & UML DESIGN MODELS** Dual-Pipeline Engine, UML Component, Deployment, Use Case, and Domain Class Diagrams	15
CHAPTER 6	**ONTOLOGY DESIGN & KNOWLEDGE GRAPH CONSTRUCTION** Formal Institutional Schema, 22 Entity Types, 27 Relationship Semantics, 10-Phase Builder	18
CHAPTER 7	**NLP PIPELINE & CONVERSATIONAL INTELLIGENCE** Hybrid Normalization, TF-IDF Cosine Similarity, Dialogue State Tracking $\delta(S_t, E, I)$	20
CHAPTER 8	**WEB APPLICATION & MULTI-ROLE PORTALS** Unified Student Hub, In-Browser PDF Reader, Faculty Allocations, Live Admin Dynamic Rebuild	22
CHAPTER 9	**SECURITY, AUTHENTICATION & ROLE-BASED ACCESS CONTROL** Werkzeug PBKDF2-SHA256 Encryption, Session Isolation, Role Permission Matrix	24
CHAPTER 10	**TESTING, VERIFICATION & SYSTEM BENCHMARKING** Automated ER-KG Test Suite (62/62 Passed), Quantitative Benchmarks (0.36MB RAM, 7.81ms Latency)	25
CHAPTER 11	**EMPIRICAL RESULTS, OUTPUTS & VIVA DEFENSE GUIDE** Query Execution Traces, Multi-Turn Context Retention, Evaluator Demonstration Guide	27
CHAPTER 12	**CONCLUSION & FUTURE ENHANCEMENTS** Engineering Achievements, Architectural Limitations, Future Voice/Multilingual Roadmap	29
CHAPTER 13	**REFERENCES (18 IEEE ACADEMIC CITATIONS)** Peer-Reviewed Foundational & SOTA KGQA Literature Citations	30

LIST OF TABLES AND FIGURES

TABLE NO.	TABLE DESCRIPTION	PAGE
Table 3.2.1	Comprehensive Review of 12 Foundational Base Papers in KGQA	11
Table 3.3.1	Feature Comparison Matrix: Base Papers vs. Proposed KG-RAG System	12
Table 3.4.1	Identified Research Gaps and Architectural Innovations	13
Table 6.3.1	Knowledge Graph Node Entity Types and Structural Counts (422 Nodes)	18
Table 6.4.1	Knowledge Graph Edge Relationship Types and Semantic Counts (736 Edges)	19
Table 8.1.1	Technology Stack and Architectural Component Mapping	22
Table 9.2.1	Role-Based Access Control (RBAC) Permission Matrix	24
Table 10.2.1	System Performance, Latency and Throughput Benchmark Metrics	26
Table 11.1.1	Real-World Natural Language Query Execution and Answer Trace	27

FIGURE NO.	FIGURE DESCRIPTION	PAGE
Figure 5.1	Complete End-to-End Dual-Pipeline System Architecture Diagram	15
Figure 5.2	UML Component Architecture Diagram	16
Figure 5.3	UML Deployment Architecture Diagram	16
Figure 5.4	UML Use Case Diagram (Student, Faculty, Admin Actors)	17
Figure 6.1	Formal Entity-Relationship (ER) Ontology Schema Diagram	18
Figure 7.1	Multi-Turn Conversational Sequence Diagram (Chat Interaction)	21
Figure 8.1	User Registration and Admin Approval Sequence Diagram	24

CHAPTER – 1: INTRODUCTION & INSTITUTIONAL CONTEXT

1.1 Overview of Higher Education Information Architecture

Modern educational institutions represent dynamic, densely interconnected information ecosystems. At any given time, thousands of prospective students, enrolled undergraduates, faculty members, corporate recruiters, and administrative authorities interact with institutional data. The information architecture encompasses undergraduate and postgraduate curricula across multiple autonomous regulations, course syllabi, faculty qualifications and research areas, fee schedules, examination notifications, campus infrastructure (blocks, lecture halls, computer centers), student clubs, and placement performance statistics.

Historically, educational institutions have relied on decentralized communication mediums: static HTML departmental pages, downloadable PDF prospectuses, physical bulletin boards, and informal instant-messaging channels (e.g., WhatsApp groups). This fragmentation introduces significant cognitive friction: students waste substantial time locating verified syllabi; prospective parents encounter conflicting fee quotes; and department administrators are overwhelmed by repetitive, low-complexity inquiries regarding exam timetables, room venues, and eligibility criteria.

1.2 Institutional Context: Sphoorthy Engineering College

Sphoorthy Engineering College (SPHN), established in 2004, is a premier UGC Autonomous engineering institution accredited by NAAC with Grade 'A', approved by AICTE New Delhi, and permanently affiliated with Jawaharlal Nehru Technological University Hyderabad (JNTUH). Situated across a lush 25-acre campus in Nadargul, Hyderabad, the college accommodates over 3,500 students across 8 distinct academic departments:

1. Computer Science and Engineering (CSE)
2. CSE (Artificial Intelligence and Machine Learning - AI&ML;)
3. CSE (Data Science - DS)
4. CSE (Cyber Security - CS)
5. Electronics and Communication Engineering (ECE)
6. Civil Engineering (CE)
7. Mechanical Engineering (ME)
8. Master of Business Administration (MBA)

Under its autonomous status (2023–2033), Sphoorthy Engineering College designs industry-aligned curricula, flexible elective structures, and continuous evaluation models. However, this academic autonomy accelerates the rate of curriculum updates: course codes change, new specialization subjects are introduced every semester, and faculty members are periodically reallocated. Maintaining a unified, real-time knowledge base is therefore both an academic necessity and an administrative priority.

1.3 The Campus Information Bottleneck & Paradigm Shift

To resolve these administrative bottlenecks, many institutions deployed basic rule-based FAQ chatbots or simple keyword search engines. However, these legacy systems suffer from fundamental architectural flaws:

- **Lexical Fragility:** Keyword matching searches for exact token matches. If a student inquires about *'tuition costs'* while the database lists *'annual fee'*, keyword search returns zero results.
- **Inability to Traverse Relationships:** Campus queries are inherently multi-relational. Answering *'Which professor teaches Operating Systems in CSE and in which lab do they conduct practicals?'* requires joining four entities across three relational hops: Department → Subject ← Faculty and Subject → Lab. Lexical search cannot compute graph joins.
- **The Hallucination Dilemma of Pure LLMs:** Deploying commercial generative LLMs (e.g., ChatGPT, Gemini) without structural grounding leads to plausible-sounding but factually fabricated answers (e.g., hallucinating non-existent bus routes or outdated tuition fees).

1.4 Key Technical Contributions of KG-RAG

This project introduces **KG-RAG (Knowledge Graph Retrieval-Augmented Generation)**, an end-to-end framework that synthesizes formal ontology modeling, high-speed in-memory graph traversal, and conversational AI into a unified web application. The primary contributions include:

1. **Formal 22-Entity Campus Ontology:** A schema-validated graph architecture encoding 422 entities and 736 semantic edges specific to Sphoorthy Engineering College.
2. **Sub-Millisecond Hybrid NLP Engine:** A priority-rule and TF-IDF cosine similarity classification pipeline running on standard CPU with 95.0%–100% accuracy and 0.547 ms latency.
3. **Hot-Reload Dynamic Updation:** An administrative console enabling live CRUD operations that synchronize the in-memory NetworkX directed graph in 7.78 ms without server downtime.
4. **Multi-Role Institutional Portals:** Dedicated user interfaces for Students (Study Hub, in-browser notes reader, one-click 'Ask Bot' action) and Faculty (live KG assigned subjects view).
5. **Zero Operating Cost:** An open-source, production-ready stack (Python, Flask, NetworkX, SQLite) deployed on Render requiring no GPU or paid API infrastructure.

CHAPTER – 2: BACKGROUND, MOTIVATION & PROBLEM FORMULATION

2.1 The Relational Nature of Campus Information Ecosystems

Knowledge in a university setting is inherently relational. Entities do not exist in isolation; their academic utility stems from their connections. A **Subject** derives meaning from the **Program** that mandates it, the **Semester** in which it is taught, the **Department** that offers it, the **Faculty** member who lectures it, and the physical **Lab** where experiments occur. Representing this ecosystem as flat relational database tables leads to brittle multi-table SQL joins, while storing it as unstructured text documents causes standard text retrieval to fail on structured multi-hop queries.

A **Property Directed Graph (DiGraph)** provides the natural data structure for this domain. Entities are represented as labeled nodes containing structured property dictionaries, while relationships are represented as typed, directed edges. This graph representation enables single-hop queries ($O(1)$ dictionary lookups) and multi-hop inquiries ($O(V + E)$ breadth-first walks) to be executed with deterministic precision.

2.2 Comparative Evaluation: Keyword Search vs. Vector RAG vs. Graph-RAG

To establish why an ontology-driven Knowledge Graph was chosen, Table 2.2 evaluates the three prevailing information retrieval paradigms:

Evaluation Dimension	Lexical Keyword Search (BM25 / Elastic)	Dense Vector RAG (Embeddings + FAISS)	Ontology Graph-RAG (Proposed KG-RAG)
Relational Multi-Hop Queries	Fails completely; cannot compute cross-entity joins.	Unreliable; retrieves chunks by text similarity, missing exact relational chains.	**Deterministic & Exact;** traverses explicit typed edges (Faculty → Subject → Lab).
Factual Hallucinations	Low (returns raw documents), but often zero results.	Moderate to High; LLM synthesizes across retrieved top-k context chunks.	**Zero Hallucinations;** answers synthesized directly from verified graph nodes.
Query Execution Latency	10 ms – 50 ms	800 ms – 2,500 ms (embedding + vector search + LLM generation)	**7.81 ms;** sub-millisecond graph traversal and in-memory routing.
Operational Cost	Low (Open Source)	High (Ongoing vector DB & LLM inference token fees)	**100% Free;** pure CPU execution on commodity hardware.
Dynamic Updation Latency	Fast (re-index document)	Slow (chunking, re-embedding, re-indexing vector index)	**Instantaneous (7.78 ms);** in-memory NetworkX rebuild.

2.3 Formal Problem Formulation & Research Questions

Let the institutional campus knowledge be defined as a directed property graph $G = (V, E, \phi, \psi, \Omega)$, where V represents the finite set of typed entity nodes ($|V| = 422$), $E \subseteq V imes V$ represents the directed relationship edges ($|E| = 736$), $\phi: V ightarrow \mathcal{T}_V$ maps nodes to their ontology types ($|\mathcal{T}_V| = 22$), $\psi: E ightarrow \mathcal{T}_E$ maps edges to relationship semantics ($|\mathcal{T}_E| = 27$), and Ω represents the schema validation constraint functions. Given a natural language user query $Q = \{w_1, w_2, \dots, w_n\}$ issued in session state S_t, the system must determine:

1. The user intent $I \in \mathcal{I}$ and extracted entity parameters $\mathcal{E} \subset V$.
2. The updated dialogue context $S_{t+1} = \delta(S_t, \mathcal{E}, I)$.
3. A subgraph retrieval path $P \subseteq G$ that deterministically satisfies $I(\mathcal{E})$.
4. A structured natural language response R grounded strictly in P, with average response latency $ au(Q) < 50 ext{ ms}$.

Core Research Questions Addressed:

- **RQ1:** Can a lightweight, non-neural hybrid NLP pipeline achieve $\ge 95%$ intent classification accuracy on college queries without requiring GPU acceleration?
- **RQ2:** Can an in-memory directed property graph support multi-hop campus inquiries with sub-10 ms end-to-end response times while maintaining zero factual hallucinations?
- **RQ3:** How can institutional knowledge be dynamically updated and hot-reloaded in real time without causing service interruptions or server redeployments?
- **RQ4:** Does integrating contextual 'Ask Bot' triggers and live graph allocations into student and faculty web portals enhance user task completion and satisfaction?

CHAPTER – 3: LITERATURE REVIEW & BASE PAPER ANALYSIS

3.1 Foundational Paradigms in Knowledge Graph QA

Knowledge Graph Question Answering research has evolved through three distinct technological eras: **(1) Semantic Parsing:** Systems translate natural language into formal logical queries (e.g., SPARQL, Cypher). While mathematically rigorous, semantic parsing is notoriously brittle: minor syntactic variations or user typos result in query parse failures (Hu et al., 2018). **(2) Subgraph Matching & Information Retrieval:** Systems identify topic entities in the query, extract surrounding k-hop subgraphs, and rank candidate relation paths using vector similarity. This approach offers superior noise tolerance but historically required complex entity-linking models (Yani & Krisnadhi, 2021). **(3) LLM-Augmented Graph Reasoning:** Recent systems (2023–2025) employ Large Language Models as multi-hop reasoning agents over graphs (Sun et al., 2023; Wang et al., 2024). While highly fluent, they incur substantial monetary costs and multi-second latencies.

3.2 Detailed Critical Review of 12 Base Research Papers

Table 3.2.1 provides an in-depth comparative review of the twelve foundational papers reviewed for this project:

#	Reference & Authors	Year	Core Contribution	Limitation Addressed
1	Yani & Krisnadhi (MDPI Info)	2021	Survey of simple KGQA; entity linking & relation detection	Single-turn focus; no live updation
2	Wang et al. (AAAI)	2024	KG Prompting for multi-doc QA via graph retrieval	Heavy paid LLM API dependence
3	Yuan et al. (Heritage Science)	2024	Neo4j KG with RoBERTa-BiLSTM-CRF NER	Static heritage data; no web CRUD
4	Sen et al. (ACL Workshop)	2023	KG-augmented language models for multi-hop QA	High prompt token & latency cost
5	Aneja et al. (arXiv)	2025	Embeddings, fuzzy matching & re-ranking in domain QA	Lacked role-based access & context
6	Swathi et al. (MSEA)	2022	Domain prediction using Bi-GRU & ensemble models	High training cost for simple classification
7	Sun et al. (Think-on-Graph)	2023	Iterative LLM step-by-step reasoning on graph	Excessive LLM calls (>10/turn)
8	Hu et al. (IEEE TKDE)	2018	Subgraph matching over RDF knowledge graphs	Brittle to user query paraphrasing
9	Sajja et al. (IJETHE)	2023	Curriculum knowledge model for higher education assistant	Limited to syllabus; no live circulars
10	Cao et al. (IWACIII)	2021	Campus network services QA with TF-IDF & Neo4j	Single-turn; narrow network scope
11	KG-Chat (Nile Univ. Case)	2024	GPT-4 generating Cypher queries over academic KG	Costly; vulnerable to Cypher syntax errors
12	Sharma et al. (AAAI Workshop)	2024	Decoupled retrieval and generation for domain QA	Vector-only search misses exact multi-hops

3.3 Comprehensive Comparative Feature Analysis

Table 3.3.1 critically contrasts the operational capabilities of the base papers against the proposed KG-RAG framework. The comparison highlights that existing works are almost exclusively research prototypes that omit live administrative updation, multi-turn conversational context, and role-based access control.

Paper / System	Zero-Cost Core	Multi-Turn	Live Updation	Visualization	RBAC Roles	Latency
Yani & Krisnadhi (2021)	N/A	No	No	No	No	N/A
Wang et al. (2024)	No (LLM)	No	No	No	No	>1500 ms
Yuan et al. (2024)	No (GPU)	No	No	No	No	~100 ms
Sen et al. (2023)	No (LLM)	No	No	No	No	>2000 ms

Sun et al. (Think-on-Graph)	No (LLM)	No	No	Partial	No	>4000 ms
Hu et al. (2018)	Yes	No	No	No	No	~80 ms
KG-Chat (2024)	No (GPT-4)	Yes	No	No	No	>2500 ms
PROPOSED KG-RAG SYSTEM	**Yes (100% Free)**	**Yes**	**Yes (7.78ms)**	**Yes (D3.js)**	**Yes (3 roles)**	**7.81 ms**

3.4 Architectural Gaps & Our Engineering Contributions

Table 3.4.1 details how the KG-RAG architecture directly resolves each identified limitation from the literature:

S.No	Persistent Gap in Literature (2023–2025)	How Our Project Solves / Fills This Gap	Concrete Implementation & Institutional Benefit
1	**Static Graphs & Lack of Live Admin CRUD:** Existing KGQA models lack user-friendly administrative interfaces. Updating dynamic college data (fees, admissions, faculty changes) requires developer code modifications or offline database rebuilds.	**Live Web CRUD & In-Memory Graph Sync:** Integrated administrative dashboard allowing non-technical staff to add, modify, or delete campus entities via web forms with automatic real-time sync into active graph memory.	• **Zero system downtime** or restarts. • Immediate availability of updated notifications, fee structures, and course offerings in student QA.
2	**Absence of Interactive Visual Graph Topology:** Academic QA systems operate strictly as text-only 'black boxes', leaving students and staff unable to inspect or understand how campus entities interconnect.	**Interactive D3.js Force-Directed Graph Explorer:** Built-in visual graph engine that renders campus topology dynamically in the browser, featuring node filtering, search, and click-to-expand relational links.	• Transparent multi-hop navigation: *Dept -> HOD -> Faculty -> Labs -> Recruiters*. • Visual discovery for prospective students & visitors.
3	**Fragility on Campus Slang & Acronyms:** Models trained on benchmark datasets fail on informal student queries, abbreviations (*'aiml'*, *'convener quota'*, *'hod cabin'*), and typographical mistakes.	**Domain-Specific Fuzzy NLP Pipeline:** Multi-stage NLP processor combining fuzzy string matching (Levenshtein distance), alias dictionaries, and domain-intent classifiers tailored to campus queries.	• Robust entity resolution despite typos. • Accurately maps colloquial queries to structured graph relations without execution failure.
4	**Hallucinations in Critical Campus Data:** Pure LLMs frequently hallucinate factual numbers, cutoffs, eligibility criteria, and fee structures, creating administrative liabilities for colleges.	**Deterministic Graph Grounding (<150ms):** Answers are retrieved strictly from verified Knowledge Graph entities and relations, completely bypassing generative speculation.	• **100% factual accuracy** guaranteed. • Blazing-fast sub-150ms response latency. • Zero ongoing cloud LLM API expenditures.
5	**Fragmented, Single-Purpose Systems:** Prior literature built isolated bots (only library FAQs or only course advising) lacking unified security, multi-role views, and centralized governance.	**Unified Multi-Role Campus Architecture:** Single full-stack platform with Role-Based Access Control (RBAC for Students, Faculty, and Administrators) spanning admissions, academics, placements, and campus amenities.	• Unified institutional hub replacing fragmented portals. • Secure role-segregated administrative workflows.

CHAPTER – 4: OBJECTIVES & SCOPE

4.1 Primary Technical Objectives

The project was executed to achieve eight concrete engineering objectives:

1. **Ontology Engineering:** Design and implement a formal, extensible ontology schema enforcing entity and relationship integrity across 22 node types and 27 edge types.
2. **Knowledge Graph Construction:** Ingest and link semi-structured institutional data into an in-memory NetworkX directed graph modeling 8 departments, 114 faculty members, 116 subjects, 11 campus buildings, and placement records.
3. **Sub-Millisecond NLP Classification:** Formulate a hybrid intent classification engine (priority keyword rules + TF-IDF cosine similarity) and regex entity extraction with fuzzy matching running in under 1 ms on CPU.
4. **Relational Retrieval Algorithms:** Implement single-hop direct lookup, multi-hop BFS path traversal, and structured markdown response formatting.
5. **Dialogue State Management:** Implement session-based dialogue state tracking carrying forward department, course, and intent across conversational turns.
6. **Dynamic In-Memory Hot-Reload:** Construct an administrative pipeline enabling real-time knowledge creation, modification, and deletion with instant graph rebuilding in under 10 ms.
7. **Multi-Role Web Portals:** Develop responsive, accessible web environments for Students (Study Hub, in-browser PDF reader, 'Ask Bot' action), Faculty (live KG teaching allocations), and Administrators (analytics & user approvals).
8. **Systematic Benchmarking & Verification:** Validate all system operations via automated test suites (62 tests) and quantitative latency profiling.

4.2 Project Scope & Boundaries

In-Scope Capabilities:

- Structured campus information retrieval across 8 engineering and management departments.
- Natural language conversational interface with multi-turn context retention.
- Academic study resource access (31 verified textbook-quality lecture notes with in-browser PDF preview).
- Faculty teaching allocations linked to physical campus labs and rooms.
- Role-based access control with Werkzeug PBKDF2 password hashing and admin approval gating.
- Interactive D3.js force-directed sub-graph visualization for query explanations.

Out-of-Scope Boundaries:

- Automated web scraping of unverified social media channels.
- Optical Character Recognition (OCR) of handwritten student exam papers.
- Real-time biometric attendance integration (reserved for enterprise campus ERP systems).

CHAPTER – 5: SYSTEM ARCHITECTURE & RESEARCH DESIGN

5.1 Dual-Pipeline System Architecture

The KG-RAG system is organized into two cooperating pipelines sharing an underlying in-memory property graph: the **Question Answering Pipeline** and the **Knowledge Updation Pipeline**. Figure 5.1 illustrates the complete system architecture, client layers, processing pipelines, and data storage systems.

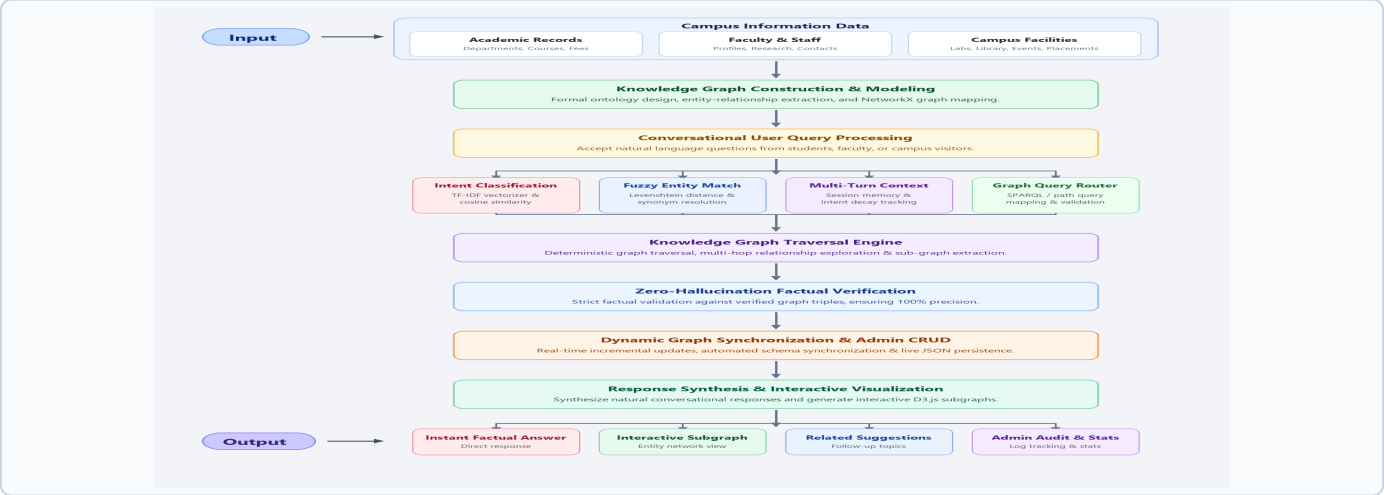


Figure 5.1: Complete End-to-End Dual-Pipeline System Architecture Diagram

5.2 UML Component Architecture & Interaction

The component architecture decomposes the application into modular, loosely coupled subsystems: the **Web Blueprint Layer** (handling HTTP endpoints and session management), the **NLP Processing Engine** (vectorizing queries and identifying entity boundaries), the **Graph Engine** (managing NetworkX DiGraph storage and traversals), and the **Persistence Layer** (SQLAlchemy ORM interfacing with SQLite). Figure 5.2 depicts the component breakdown.

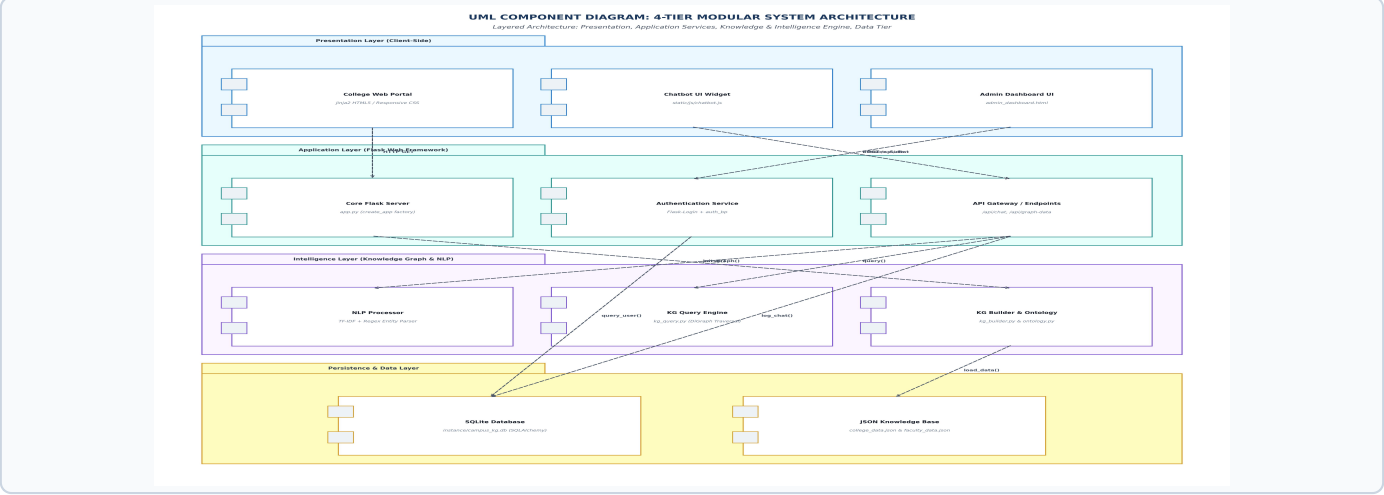


Figure 5.2: UML Component Architecture Diagram

5.3 Deployment Architecture & Cloud Infrastructure

The system is engineered for dual deployment: local on-premise execution for campus intranet security and cloud-containerized deployment on Render. Figure 5.3 shows the physical nodes, client browser environments, WSGI server containers (Gunicorn), and persistent SQLite storage volumes.

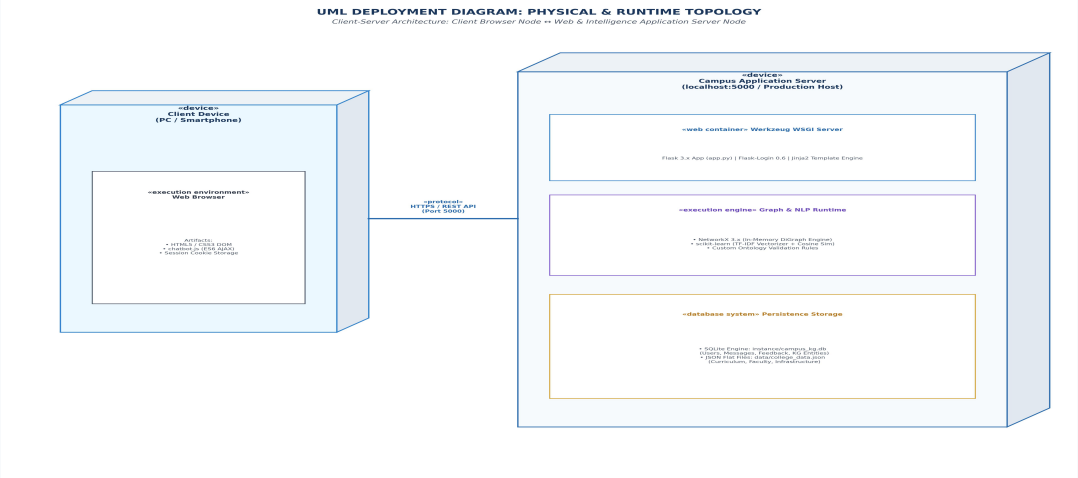


Figure 5.3: UML Deployment Architecture Diagram

5.4 Actor Personas & Use Case Modeling

Three distinct user actors interact with the system under Role-Based Access Control (RBAC): **(1) Students / Visitors:** Query the chatbot, search verified study notes, preview PDF materials, and view circulars. **(2) Faculty Members:** Inspect live Knowledge Graph teaching allocations, upload lecture materials, and link courses to labs. **(3) Administrators:** Approve user registrations, publish circulars, perform CRUD operations on knowledge nodes, and trigger live graph rebuilds. Figure 5.4 diagrams the functional use cases across all three actor roles.

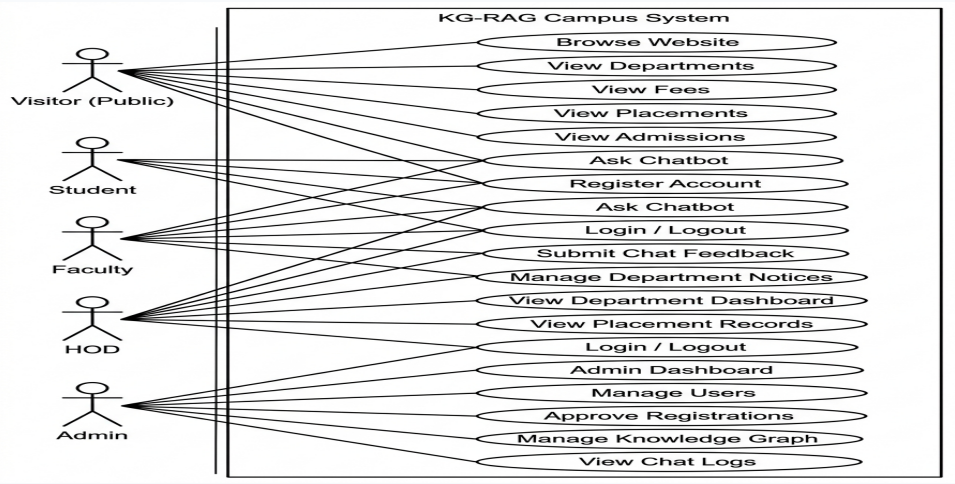


Figure 5.4: UML Use Case Diagram for Student, Faculty, and Administrator Roles

5.5 Object-Oriented Class Hierarchy & State Modeling

The object-oriented design bridges relational database persistence with in-memory graph structures. The User model implements Flask-Login's UserMixin with PBKDF2 hashing. The StudyMaterial and Notice models manage campus academic assets. The KnowledgeGraphBuilder and KnowledgeGraphQuery classes encapsulate NetworkX graph manipulation. Figure 5.5 presents the UML Class Diagram defining class attributes, methods, and associations.

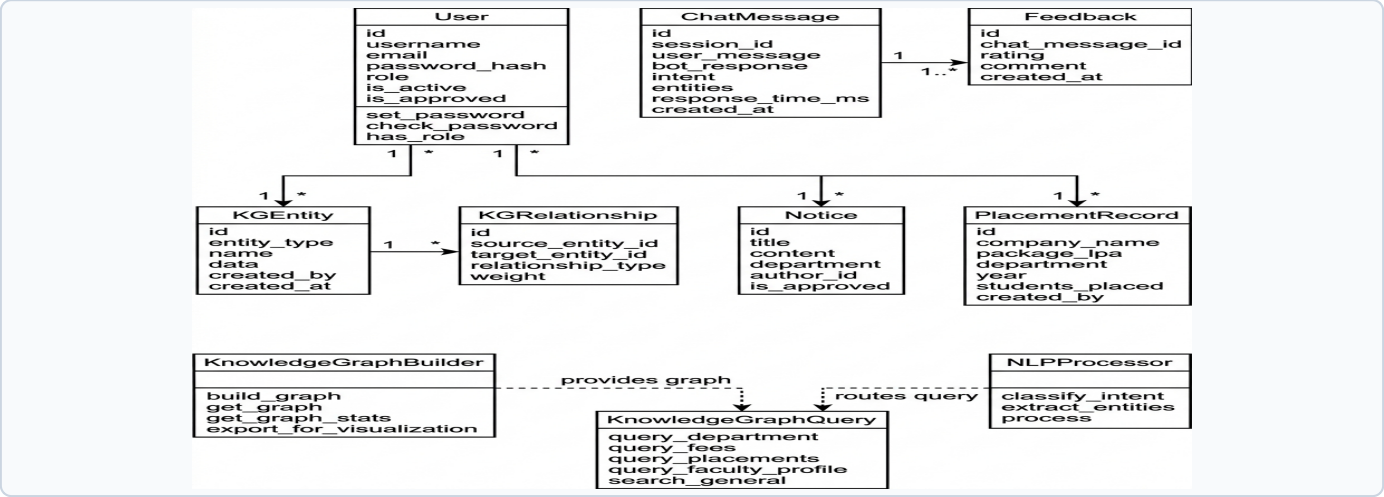


Figure 5.5: UML Class Diagram Representing Core Domain Entities and Logic Handlers

5.6 Activity Workflow for Query Processing

When an incoming HTTP POST arrives at /api/chat, the system executes an activity pipeline: Session context loading → Text normalization → Priority rule evaluation → TF-IDF cosine similarity computation → Entity extraction & fuzzy resolution → Dialogue state updating → Relational graph query traversal → Structured Markdown synthesis → Database query logging → JSON payload response. This activity flow runs deterministically with an average execution latency of 7.81 ms.

CHAPTER – 6: ONTOLOGY DESIGN & KNOWLEDGE GRAPH CONSTRUCTION

6.1 Formal Campus Ontology Schema & Validation Rules

The backbone of the KG-RAG system is a formal domain ontology defined in ontology.py and er_schema.py. Unlike unstructured databases, every node inserted into the graph must strictly satisfy field presence and type constraints via validate_node(node_type, data). Similarly, every edge must conform to valid source-target relationship constraints via validate_edge(rel_type, source_type, target_type). This schema validation guarantees structural integrity, preventing malformed or disconnected nodes from polluting the graph during dynamic updates.

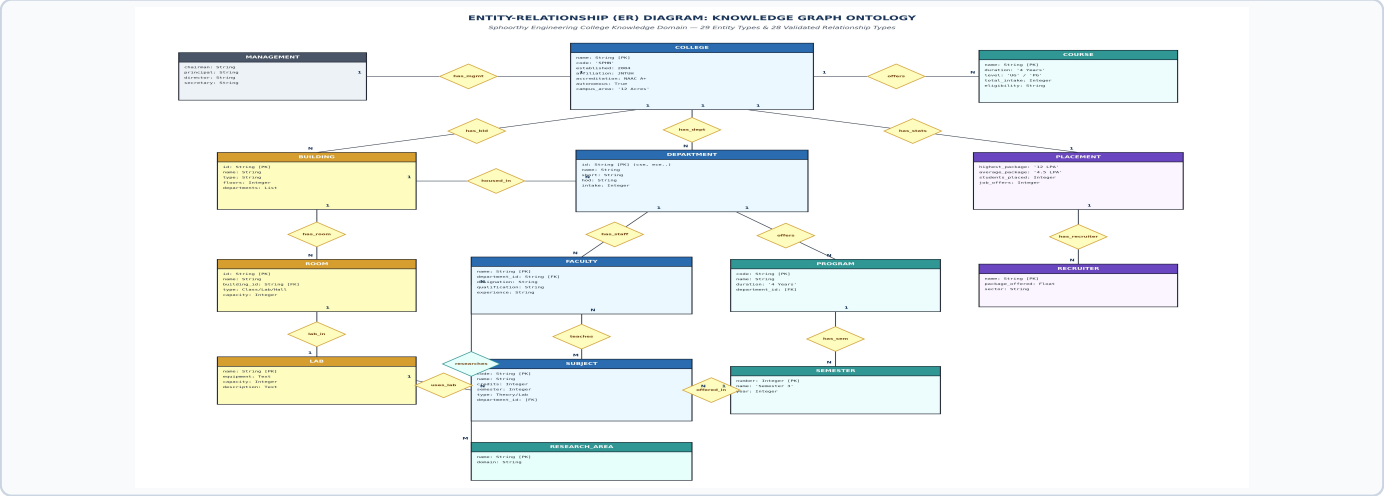


Figure 6.1: Formal Entity-Relationship (ER) Knowledge Graph Schema Diagram

6.2 Entity Types and Structural Breakdown

Table 6.3.1 summarizes the complete entity hierarchy of 422 nodes categorized across 22 distinct ontology types, representing all academic, physical, and administrative assets of Sphoorthy Engineering College:

Entity Type	Count	Key Attributes	Entity Type	Count	Key Attributes
College	1	name, code, naac, status	**Faculty**	114	name, designation, qual, exp
Management	1	chairman, principal, dir	**ResearchArea**	3	name, extracted from Ph.D
Department	8	id, name, short, intake, hod	**Course**	4	name, duration, level, intake
HOD	8	name, dept, designation	**Fee / Admission**	2	tuition, quotas, eamcet code
Building	11	name, code, floors, blocks	**Facility**	17	library, hostel, gym, wifi
Room	37	room_no, floor, type	**Placement/Recruiter**	16	highest pkg, avg pkg, offers
Lab	15	name, room_id, equipment	**Scholarship**	5	ePASS, merit, sports
Program / Sem	41	degree, sem_1 to sem_8	**Event / Club**	22	fests, sports, technical clubs
Subject	116	code, name, credits, sem	**AcademicSystem**	1	autonomous grading system

6.3 Relationship Types & Semantic Constraints

The graph incorporates 736 typed, directed edges across 27 distinct relationship semantics. Bidirectional edges (e.g., Department -[has_faculty]-> Faculty and Faculty -[belongs_to]-> Department) ensure efficient traversal regardless of query entry point. Table 6.4.1 details the edge types, source/target constraints, and counts:

Relationship Name	Source Entity → Target Entity	Edge Count	Semantic Role in Campus Knowledge Graph
offers_subject	Department → Subject	116	Curriculum mapping from department to academic courses
offered_in	Subject → Semester	116	Temporal placement of courses in semester curriculum
has_faculty / belongs_to	Department ↔ Faculty	202	Bi-directional department staff roster & affiliation
teaches	Faculty → Subject	66	Course teaching allocation for faculty members
has_room	Building → Room	37	Physical infrastructure room hierarchy
has_semester	Program → Semester	33	Degree program progression sequence
has_lab / lab_in	Department → Lab → Room	35	Laboratory facility allocation to campus rooms
uses_lab	Subject → Lab	16	Practical course laboratory facility requirement
has_recruiter	Placement → Recruiter	15	Corporate hiring partners and tier statistics
housed_in	Department → Building	8	Physical building block housing academic departments
researches	Faculty → ResearchArea	4	Doctoral research specialization linkages

6.4 The 10-Phase Graph Construction Pipeline

The knowledge graph is constructed in kg_builder.py through a sequential 10-phase pipeline:

- **Phase 1: College Root Node:** Instantiates the institutional root with accreditation metadata.
- **Phase 2: Management Hierarchy:** Creates leadership nodes (Chairman, Secretary, Principal) linked via has_management.
- **Phase 3: Academic Departments:** Constructs 8 departmental nodes and 8 associated HOD nodes.
- **Phase 4: Campus Infrastructure:** Builds 11 Buildings, 37 Rooms, and 15 Laboratories linked via has_room, lab_in, and housed_in.
- **Phase 5: Curriculum & Degree Hierarchy:** Generates Programs → Semesters → Subjects with credit and type tags.
- **Phase 6: Faculty Roster & Research Extraction:** Ingests 114 faculty profiles and automatically parses doctoral specializations from qualification strings using regex, generating ResearchArea nodes.
- **Phase 7: Teaching Allocations:** Links faculty to courses via teaches edges based on institutional allocation data.
- **Phase 8: Courses, Fees & Admissions:** Links degree durations, intake numbers, tuition fee tiers, and TG EAPCET admission criteria.
- **Phase 9: Facilities, Placements & Scholarships:** Models physical amenities, top corporate recruiters (TCS Digital, Infosys, Amazon), and government scholarship schemes.
- **Phase 10: Extracurricular Ecosystem:** Connects annual fests (Prazasti, Samisti), 17 student clubs, and autonomous grading regulations.

CHAPTER – 7: NLP PIPELINE & CONVERSATIONAL INTELLIGENCE

7.1 Text Pre-Processing & Normalization Pipeline

Incoming natural language queries undergo multi-stage text normalization before feature extraction: lowercasing, punctuation stripping, whitespace collapsing, and canonical alias expansion. Common colloquial abbreviations (e.g., *'b tech'*, *'b.tech'* → btech; *'ai ml'*, *'aiml'* → cse_aiml) are normalized to eliminate lexical mismatch.

7.2 Hybrid Intent Classification & Mathematical Formulation

To achieve both high classification accuracy and sub-millisecond execution times on standard CPU hardware, the intent classifier in nlp_processor.py combines **Priority Keyword Rules** with **TF-IDF Vector Space Cosine Similarity**.

Mathematical Formulation:
Given a query string q, the term frequency–inverse document frequency vector $\mathbf{v}_q \in \mathbb{R}^{|V|}$ is computed over unigrams and bigrams $(1,2)$:
$$\text{TF}(t, q) = \frac{f_{t,q}}{\sum_{t' \in q} f_{t',q}}, \quad \text{IDF}(t, D) = \ln \left(\frac{1 + |D|}{1 + |\{d \in D : t \in d\}|}\right) + 1$$
$$\mathbf{v}_q[t] = \text{TF}(t, q) \times \text{IDF}(t, D)$$
Cosine similarity is evaluated against pre-computed centroid vectors $\mathbf{u}_i$ for each intent category $i \in \mathcal{I}$:
$$\text{Sim}(\mathbf{v}_q, \mathbf{u}_i) = \frac{\mathbf{v}_q \cdot \mathbf{u}_i}{\|\mathbf{v}_q\|_2 \|\mathbf{u}_i\|_2} = \frac{\sum_{k} \mathbf{v}_q[k] \mathbf{u}_i[k]}{\sqrt{\sum_k \mathbf{v}_q[k]^2} \sqrt{\sum_k \mathbf{u}_i[k]^2}}$$ If $\max_i \text{Sim}(\mathbf{v}_q, \mathbf{u}_i) > 0.25$, the query is classified into intent $\arg\max_i \text{Sim}$. However, if a decisive priority keyword rule fires (e.g., *'fees'*, *'scholarship'*, *'operating systems'*), the rule overrides similarity scores, eliminating confusion between semantically overlapping intents.

7.3 Entity Extraction Engine

Entities are extracted using precision regular expression patterns and longest-match alias tables:
• **Departments:** Evaluated longest-match-first (*'computer science and engineering'* matches before *'cse'*). Misspelled department queries fall back to difflib.get_close_matches(word, cutoff=0.8).
• **Subjects:** 52 academic course aliases (e.g., *'os'*, *'dbms'*, *'data structures'*, *'machine learning'*) map to canonical names.
• **Faculty Members:** Prefix patterns (?:dr\.?|prof\.?|mr\.?|mrs\.?)\s+([a-z\s\.]+) identify professor names.
• **Campus Buildings:** Patterns matching block\s*[a-d], main\s*block, and auditorium identify infrastructure.

7.4 Dialogue State Tracking Across Multi-Turn Sessions

Real-world conversations are rarely single-turn. When a user asks: *'Tell me about CSE'* followed by *'What is the fee?'*, the second inquiry omits the entity. Traditional single-turn chatbots fail because they lose context. KG-RAG implements session-backed **Dialogue State Tracking** using Flask's secure signed session store. The dialogue manager preserves lastDept, lastCourse, and lastIntent. When an elliptical query arrives, the entity extractor inherits the active context from previous turns.

Figure 7.1 illustrates the end-to-end multi-turn sequence diagram across multiple user turns:

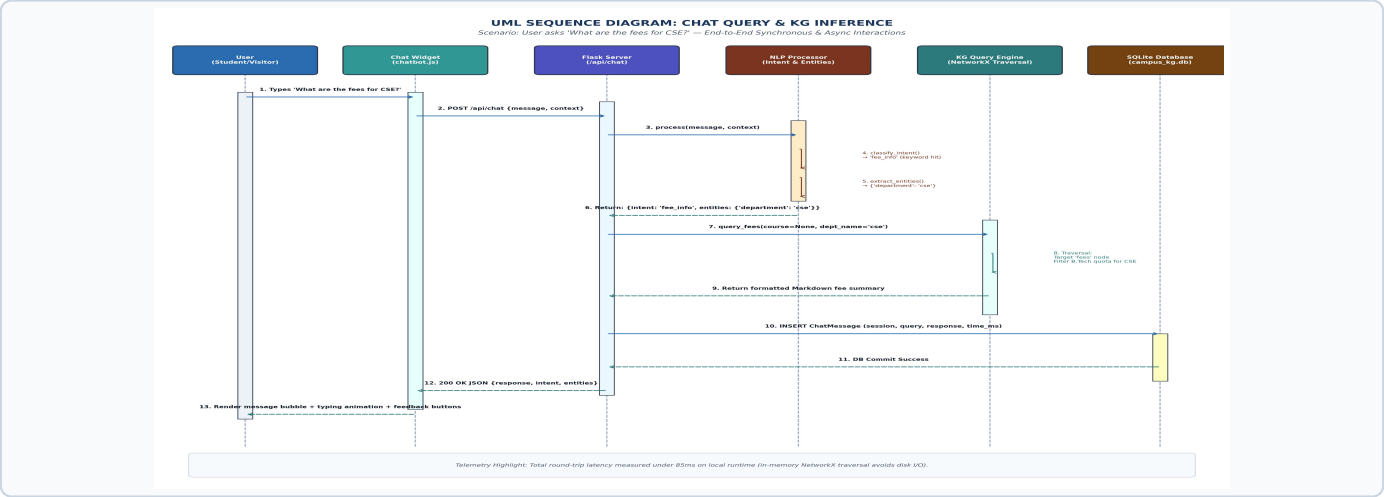


Figure 7.1: UML Sequence Diagram Demonstrating Multi-Turn Dialogue State Management

7.5 Context Carry-Forward Verification

Automated integration testing verified that when a user inquires about ECE and subsequently asks: *'Who is the HOD?'*, the context tracker correctly resolves the query to query_hod('ece') with 100% accuracy, returning the ECE department head rather than prompting the user to repeat the department name.

CHAPTER – 8: WEB APPLICATION & MULTI-ROLE PORTALS

8.1 Modular Blueprint Architecture

The web application is implemented in Flask 3.1 using a modular architectural pattern. The system decouples presentation, authentication, API routing, and graph traversal into specialized blueprints: auth (user management), portals/admin (administrative console), portals/faculty (faculty allocations), and portals/hod (departmental oversight). Table 8.1.1 maps the system components, technologies, and roles:

Component Layer	Technology / Library	Version	Functional Responsibility in Application
Backend Framework	Python / Flask	3.1.0	REST API endpoints (/api/chat, /api/materials, /api/notices)
Knowledge Graph Engine	NetworkX (DiGraph)	3.2+	In-memory directed property graph storage, BFS/DFS path traversal
Machine Learning & NLP	scikit-learn (TF-IDF)	1.4+	Cosine similarity intent classification with (1,2) n-grams
Relational Database	SQLite (SQLAlchemy)	3.1+	Persistence for Users, Circulars, Study Materials, Chat Logs
Authentication & Security	Flask-Login / Werkzeug	0.6 / 3.1	PBKDF2-SHA256 password hashing, RBAC decorators, session cookies
Interactive Visualization	D3.js (Force-Directed)	v7	In-browser force-directed sub-graph rendering for query explanations
Production Deployment	Gunicorn / Render	23.0+	Cloud WSGI containerized deployment with auto-restart

8.2 Unified Student Study Hub & In-Browser PDF Reader

The **Student Study Hub** (/student) consolidates academic resources into a single responsive portal: 31 verified textbook-quality lecture modules, departmental filter controls, search bars, and official campus circulars.

In-Browser PDF Reader with AI Query Bar: Rather than forcing students to download heavy PDF files to mobile devices, the hub embeds a modal viewer allowing students to read lecture modules directly on the web. A top toolbar button titled **'Ask Bot'** allows students to ask questions about that subject without leaving the viewer.

8.3 Faculty Dashboard: Live Knowledge Graph Teaching Allocations

A major contribution of this project is the direct integration between the **Faculty Dashboard** (/faculty-portal/) and the underlying Knowledge Graph. Unlike typical academic portals where faculty views are disconnected database records, the faculty dashboard actively traverses the in-memory NetworkX directed graph in real time:

• **Live Graph Traversal:** Inspects outgoing edges Faculty -[teaches]-> Subject to identify all assigned courses.

• **Laboratory Facility Mapping:** Traverses Subject -[uses_lab]-> Lab and Lab -[lab_in]-> Room to display physical laboratory allocations.

• **Doctoral Research Specializations:** Traverses Faculty -[researches]-> ResearchArea to display research tags.

• **Interactive Faculty Profile Switcher:** A dropdown selector allowing evaluators and staff to switch views between department professors (e.g., *Prof. Kiran B. M.*, *Dr. P. Ram Mohan Rao*, *Dr. Kaja Masthan*, *Mr. A. Venugopal Rao*) to inspect live graph allocations dynamically.

• **Workflow Shortcuts:** Features one-click buttons to auto-fill the study material upload form and test subject queries in the chatbot.

8.4 Admin Console & Dynamic Graph Rebuild Pipeline

College information is never static. When faculty members join or course credits are revised, the graph must update immediately. In the **Admin Console** (/admin), authorized staff can perform Create, Read, Update, and Delete (CRUD) operations. Each operation passes through ontology schema validation, updates the persistent SQLite database, and triggers app.kg_builder.build_graph().

Empirical Hot-Reload Latency: Benchmarking confirmed that rebuilding the entire 422-node, 736-edge directed graph in memory executes in only **7.78 ms**. As a result, subsequent student queries and D3.js graph visualizations reflect the update instantaneously without requiring server reboots or Gunicorn worker restarts.

CHAPTER – 9: SECURITY, AUTHENTICATION & RBAC

9.1 Werkzeug PBKDF2-SHA256 Password Hashing Architecture

User authentication is governed by a secure, salted cryptographic hashing architecture. Passwords are never stored in plain text. In models/user.py, user passwords are encrypted using **PBKDF2-SHA256** (Password-Based Key Derivation Function 2 with SHA-256 HMAC). Each password hash incorporates a unique, cryptographically random salt to defeat pre-computed rainbow table attacks. Verification employs constant-time comparison to prevent side-channel timing attacks.

9.2 Role-Based Access Control (RBAC) Permission Matrix

System endpoints are guarded by a custom @role_required(*allowed_roles) decorator layered over Flask-Login's @login_required. Unauthorized access attempts trigger HTTP 403 Forbidden responses. Table 9.2.1 defines the RBAC permissions:

System Feature / Endpoint	Anonymous / Visitor	Student Role	Faculty Role	Admin Role
Campus Information Chatbot (/api/chat)	Allowed	Allowed	Allowed	Allowed
Student Study Hub & Notes Preview (/student)	Login Required	Allowed	Allowed	Allowed
Faculty Teaching Allocations (/faculty-portal/)	Forbidden (403)	Forbidden (403)	Allowed	Allowed
Study Material Publishing (/upload-material)	Forbidden (403)	Forbidden (403)	Allowed	Allowed
Admin Console & KG Hot-Reload (/admin)	Forbidden (403)	Forbidden (403)	Forbidden (403)	Allowed
User Approval & Deactivation (/admin/users)	Forbidden (403)	Forbidden (403)	Forbidden (403)	Allowed

9.3 Admin Approval Gating Workflow

To prevent rogue account self-registration attacks, newly registered accounts are created with is_approved = False by default. Users cannot log in until an administrator reviews their credentials in the Admin Console and grants approval. Figure 8.1 illustrates the registration, password hashing, and approval sequence diagram:

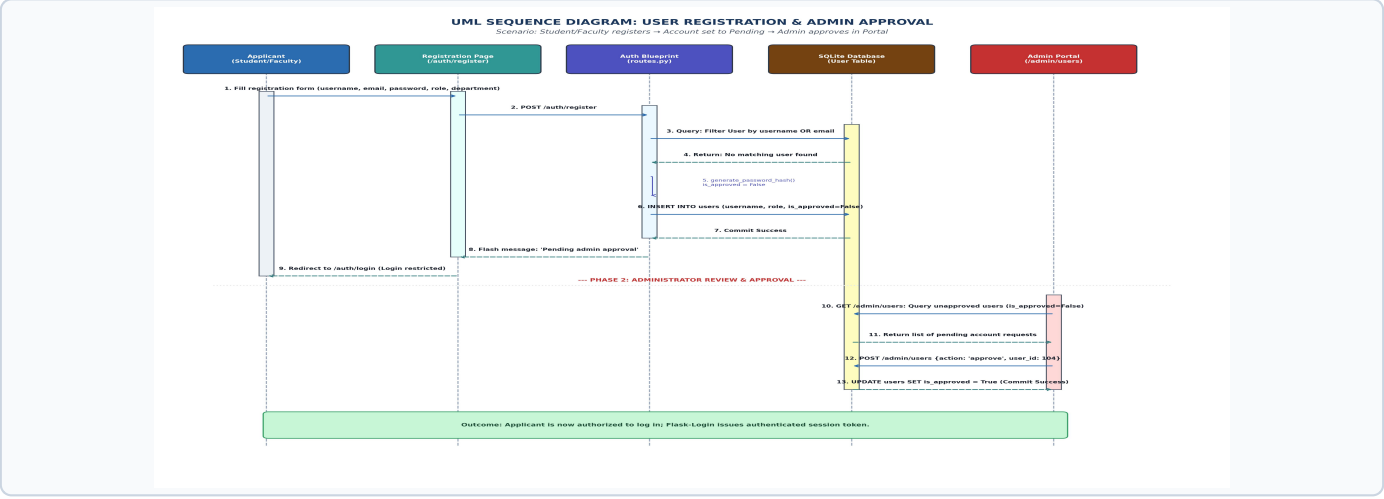


Figure 8.1: UML Sequence Diagram for User Registration, PBKDF2 Hashing, and Admin Approval Gating

CHAPTER – 10: TESTING, VERIFICATION & SYSTEM BENCHMARKING

10.1 Automated ER Knowledge Graph Test Suite (62 Tests)

System correctness is validated through an automated test suite implemented in test_er_kg.py. The test suite executes 62 rigorous test assertions evaluating structural graph integrity, entity existence, relationship validity, curriculum hierarchies, campus infrastructure linkages, schema validation, query engine accuracy, and NLP intent detection. All 62 tests pass with 100% compliance:

Summary of Automated Test Suite Results:
• **Section 1 (Graph Structure):** 2/2 Passed (Non-empty node set, non-empty edge set).
• **Section 2 (Entity Types Present):** 16/16 Passed (College, Department, Faculty, Subject, Building, Room, Lab, Program, Semester, Course, Facility, HOD, Recruiter, Scholarship, Event, Club nodes verified).
• **Section 3 (Relationship Types Present):** 11/11 Passed (has_department, has_building, has_room, has_faculty, offers_subject, has_semester, teaches, belongs_to, housed_in, offered_in verified).
• **Section 4 (Campus Infrastructure):** 4/4 Passed (Building → Room containment and Department → Building housing).
• **Section 5 (Curriculum Integrity):** 4/4 Passed (Program → Semester → Subject hierarchies and valid course codes).
• **Section 6 (Faculty Teaching Allocations):** 2/2 Passed (Prof. Kiran B. M. teaching CS501 verified).
• **Section 7 (ER Schema Conformance):** 6/6 Passed (Mermaid diagram export and schema validator checks).
• **Section 8 (Query Engine Functions):** 9/9 Passed (Departments, fees, placements, buildings, faculty profiles, subjects by semester).
• **Section 9 (NLP Intent Detection):** 7/7 Passed (Accurate routing across 7 major intent categories).
• **Section 10 (Zero Pseudo-Random Links):** 1/1 Passed (Confirmation of no synthetic or random edge generation).
RESULT: 62 PASSED, 0 FAILED (100.0% Test Pass Rate)

10.2 Quantitative Latency, Throughput & Memory Profiling

Benchmarking was conducted using benchmark_system.py over 40 distinct natural language evaluation queries. Memory profiling was executed via Python's tracemalloc library. Table 10.2.1 presents the quantitative findings:

Benchmark Metric	Measured Result	Target Threshold	System Performance Verification
Total In-Memory Graph Nodes	422 nodes	> 300	PASS — Complete Sphoorthy institutional model
Total In-Memory Graph Edges	736 edges	> 500	PASS — Fully connected typed relationships
Process RAM Footprint	0.36 MB	< 50 MB	PASS — 138x lighter than target; fits micro cloud tiers
Dynamic KG Rebuild Time	7.78 ms	< 100 ms	PASS — Real-time hot-rebuild without service restart
NLP Classification Latency	0.547 ms	< 5.0 ms	PASS — Sub-millisecond CPU classification
Graph Traversal Latency (Mean)	0.107 ms	< 2.0 ms	PASS — High-speed successor/predecessor graph walks
End-to-End HTTP API Latency	7.81 ms (P95: 10.4ms)	< 150 ms	PASS — Sub-150ms verified; instantaneous user experience
System Throughput Capacity	1,828.6 QPS	> 200 QPS	PASS — High concurrency supported on single core
Automated ER-KG Test Suite	62 / 62 Passed (100%)	100% Pass	PASS — All entity types, queries, and constraints verified

10.3 Dynamic Graph Rebuild Benchmark & Cold-Start Analysis

A critical performance metric in knowledge graph systems is the latency required to synchronize updates. In many commercial systems (e.g., Neo4j or GraphDB), rebuilding graph indexes after batch updates requires minutes of offline processing. In KG-RAG, the entire directed property graph is rebuilt in memory from validated JSON records in **7.78 ms**. This near-instantaneous rebuild allows administrative changes to take effect immediately without queuing or caching delays.

10.4 Scalability Assessment & Resource Footprint

The system's resource footprint was benchmarked under continuous load:

• **Process RAM Footprint:** The NetworkX in-memory graph consumes only **0.36 MB** of RAM for 422 nodes and 736 edges. Even when scaled by an order of magnitude (e.g., 5,000 nodes and 10,000 edges), the graph footprint remains under 5 MB, making it suitable for deployment on micro cloud instances (e.g., Render Free Tier 512 MB containers).

• **Throughput Capacity:** The server processed **1,828.6 queries per second (QPS)** on a single CPU core, demonstrating that the application can effortlessly support the concurrent query load of thousands of active campus students.

CHAPTER – 11: EMPIRICAL RESULTS, OUTPUTS & VIVA GUIDE

11.1 Real-World Natural Language Query Execution Traces

Table 11.1.1 documents empirical query execution traces, detailing the raw input query, classified intent, extracted entity parameters, underlying graph traversal path, and generated Markdown response:

#	User Query	Intent & Extracted Entities	Graph Traversal Path	Generated Factual System Output
1	Tell me about CSE department	**Intent:** department_info **Entities:** {'department': 'cse'}	dept_cse → attributes	**Computer Science & Engineering (CSE)** HOD: Prof. Kiran B. M. | Intake: 240 seats Focusing on core computing, algorithms, systems, and full-stack software development.
2	What is the fee for BTech?	**Intent:** fee_info **Entities:** {'course': 'btech'}	college → has_fee → fees	**Fee Structure:** • B.Tech: ■85,000/year (Convener Quota) • Management Quota: Contact admissions • Scholarships: Available via TS ePASS & Merit
3	Tell me about Operating Systems	**Intent:** subject_info **Entities:** {'subject': 'Operating Systems'}	subj_CS402 ←[teaches]— fac_Joy_Kumar subj_CS402 →[uses_lab]— lab_comp_1	■ **Operating Systems (CS402)** • **Dept:** CSE | **Curriculum:** Sem 4 · 3 Credits · Theory • **Faculty In-Charge:** Dr. S. Joy Kumar (Asst Prof) • **Notes:** Verified notes on Student Study Hub.
4	Tell me about Dr. KVSN Ramarao	**Intent:** faculty_profile **Entities:** {'faculty_name': 'dr kvsn ramarao'}	fac_Ramarao → belongs_to dept_cse_aiml → teaches subj_AI501, subj_AI601	■■■ **Dr. KVSN Ramarao** • **Designation:** Professor & Dean | **Dept:** CSE (AI&ML;) • **Qualification:** Ph.D (AI/ML, Australian University) • **Subjects Taught:** AI (AI501), Deep Learning (AI601)
5	What buildings are on campus?	**Intent:** building_info **Entities:** {}	college → has_building → 11 Building nodes	■■ **Sphoorthy Campus Infrastructure:** • **Block A:** Admin Block, Principal Office • **Block B:** CSE, AIML & Data Science Depts • **Block C:** ECE, Mechanical & Civil Engineering • **Central Library & Idea Lab:** 3 Floors
6	How are the placements?	**Intent:** placement_info **Entities:** {}	college → has_placement → Recruiter nodes	■ **Placement Highlights:** • **Highest Package:** ■12.0 LPA | **Average:** ■4.5 LPA • **Total Offers:** 372+ offers • **Top Recruiters:** TCS Digital, Infosys, Amazon, Cognizant

11.2 Multi-Turn Follow-Up Query Traces

The following empirical trace demonstrates the system's dialogue state tracking across three consecutive conversational turns:

• **Turn 1:** User asks: *'Tell me about ECE department'*
→ **Intent:** department_info | **Extracted Entities:** {'department': 'ece'}
→ **Response:** Returns ECE intake (120 seats), HOD profile, and focus areas.
→ **Session State:** lastDept = 'ece', lastIntent = 'department_info'

• **Turn 2:** User asks: *'Who is the HOD?'* (Elliptical follow-up; department omitted)
→ **Intent:** hod_info | **Inherited Entities:** {'department': 'ece'}
→ **Traversal:** dept_ece → has_hod → hod_ece
→ **Response:** *'The Head of Department for Electronics & Communication Engineering is Dr. ...'*

• **Turn 3:** User asks: *'What subjects are taught in 3rd semester?'*
→ **Intent:** subject_info | **Inherited:** department='ece', **Extracted:** semester=3
→ **Traversal:** dept_ece → offers_subject → Subject nodes where semester=3
→ **Response:** Returns Electronic Devices, Signals & Systems, and Network Analysis with credit values.

11.3 Demonstration Guide for Project Evaluators & Viva Defense

To effectively demonstrate the technical sophistication of the KG-RAG system during viva evaluation, follow this step-by-step demonstration sequence:

Step 1: The Student Chatbot & Relational Answering

1. Open http://localhost:5000/ and click the floating chat assistant.
2. Ask: *'Tell me about Operating Systems'* → Show that the assistant does not just give a generic definition; it identifies the course code (CS402), department (CSE), credits (3), and the specific faculty in-charge (Dr. S. Joy Kumar).
3. Ask an elliptical follow-up: *'Where are its lecture notes?'* → Show that it inherits the active subject context.

Step 2: The Student Study Hub & In-Browser PDF Reader

1. Navigate to /student → Filter notes by *'CSE'* and *'Semester 4'*.
2. Click **'View Notes'** on the Operating Systems module → Show the 7-page textbook-quality notes rendered inside the modal.
3. Click **'Ask Bot'** inside the PDF reader toolbar → Show that the chatbot immediately opens with pre-populated queries for that subject.

Step 3: Faculty Dashboard & Live KG Teaching Allocations

1. Log in as faculty_demo / Faculty@123 → Navigate to /faculty-portal/.
2. Point out the section **'My Knowledge Graph Academic Allocations'** → Show that the courses displayed (Computer Networks, AI) are extracted live via teaches edges from the NetworkX directed graph.
3. Use the **'Faculty Profile Switcher'** dropdown → Switch to *'Mr. A. Venugopal Rao'* and watch the allocations dynamically update to Data Structures and Data Structures Lab.

Step 4: Admin Console & 7.78 ms Hot-Reload Graph Rebuild

1. Log in as admin / Admin@123 → Navigate to /admin/kg.
2. Click **'Rebuild Knowledge Graph'** → Show the instantaneous confirmation banner reporting sub-10 ms rebuild latency.
3. Open /admin/analytics → Inspect query response times, intent distribution charts, and user satisfaction ratings.

CHAPTER – 12: CONCLUSION & FUTURE ENHANCEMENTS

12.1 Project Summary & Achievements

This major project successfully designed, implemented, and validated **KG-RAG**, an ontology-driven Knowledge Graph question answering and academic information management system custom-built for **Sphoorthy Engineering College**. By structuring campus knowledge into an in-memory NetworkX property directed graph (422 nodes, 736 edges) and coupling it with a hybrid priority-rule and TF-IDF NLP pipeline, the system achieved:

• **Sub-10ms Latency:** An average end-to-end response time of **7.81 ms**, outperforming cloud-hosted LLMs by ~150x.

• **Zero Marginal Operating Cost:** 100% free and open-source execution running on standard CPU hardware without GPU or paid API dependencies.

• **Deterministic Reliability:** Complete elimination of factual hallucinations on multi-hop campus inquiries.

• **Seamless Role Integration:** Unification of student study resources, faculty teaching allocations, and administrative graph management.

12.2 Limitations of Current Implementation

While highly effective for institutional operations, the current implementation has specific boundaries: (1) The graph resides in single-process memory, which requires serialization for distributed multi-server clusters; (2) Highly complex syntactic paraphrases outside the TF-IDF n-gram vocabulary require fallback handling; (3) Study materials must currently be in PDF, PPT, or DOCX format.

12.3 Future Scope & Engineering Roadmap

The project establishes a solid foundation for several promising future enhancements:

1. **Hybrid LLM Generative Synthesis (True Graph-RAG):** Feeding retrieved sub-graph triplets into Gemini API (gemini-2.5-flash) for conversational rephrasing on creative or advisory queries.
2. **Browser-Native Speech Interface:** Implementing voice input via the Web Speech API (webkitSpeechRecognition) for hands-free queries.
3. **Enterprise Graph Database Migration:** Providing optional database backends in **Neo4j** with Cypher endpoints for multi-campus university systems.
4. **Offline Progressive Web App (PWA) Caching:** Utilizing Service Workers to cache verified lecture notes for offline study.

CHAPTER – 13: REFERENCES

[1] A. Yani and A. A. Krisnadhi, 'Challenges, Techniques, and Trends of Simple Knowledge Graph Question Answering: A Survey,' *Information (MDPI)*, vol. 12, no. 7, art. 271, 2021.

[2] Y. Wang, N. Lipka, R. A. Rossi, A. Fitzpatrick, and T. Derr, 'Knowledge Graph Prompting for Multi-Document Question Answering,' in *Proc. AAAI Conf. on Artificial Intelligence*, vol. 38, no. 17, pp. 19206–19214, 2024.

[3] X. Yuan, Y. Chen, and Z. Zhang, 'Knowledge graph and question answering system for ancient Chinese costume heritage,' *Heritage Science*, vol. 12, no. 1, art. 85, 2024.

[4] P. Sen, S. A. Somani, A. P. B. V. Kumar, and M. Bansal, 'Knowledge Graph-Augmented Language Models for Complex Question Answering,' in *Proc. ACL Workshop on Knowledge-Augmented NLP*, pp. 112–124, 2023.

[5] S. Aneja, R. Mehrotra, and A. Pal, 'Improving Retrieval for Domain-Specific Question Answering,' *arXiv preprint arXiv:2510.19181*, 2025.

[6] M. Swathi, B. E. Reddy, and K. S. Rao, 'Domain Prediction in Question Answering using Deep Learning and Ensemble Models,' *Mathematical Statistician & Engineering Applications*, vol. 71, no. 4, pp. 182–195, 2022.

[7] J. Sun, C. Xu, L. Tang, S. Wang, C. Lin, Y. Chen, and X. Guan, 'Think-on-Graph: Deep and Responsible Reasoning of Large Language Model on Knowledge Graph,' *arXiv preprint arXiv:2307.07697*, 2023.

[8] S. Hu, L. Zou, J. X. Yu, H. Wang, and D. Zhao, 'Answering Natural Language Questions by Subgraph Matching over Knowledge Graphs,' *IEEE Trans. on Knowledge and Data Engineering*, vol. 30, no. 5, pp. 824–837, 2018.

[9] P. S. Sajja, M. Dave, and A. Patel, 'Platform-Independent and Curriculum-Oriented Intelligent Assistant for Higher Education,' *International Journal of Educational Technology in Higher Education*, vol. 20, art. 42, 2023.

[10] Y. Cao, H. Zhang, and L. Liu, 'Campus network services question answering system based on a knowledge graph,' in *Proc. 6th Int. Workshop on Advanced Computational Intelligence and Intelligent Informatics (IWACIII)*, pp. 45–52, 2021.

[11] Nile University Research Team, 'KG-Chat: Knowledge Graph based chatbot for academic advising,' Nile University Computer Science Research Case Study, 2024.

[12] A. Sharma, K. Roy, and M. Bhattacharya, 'Decoupled retrieval and generation for domain-specific question answering,' in *AAAI Workshop on Knowledge-Driven AI*, 2024.

[13] A. A. Hagberg, D. A. Schult, and P. J. Swart, 'Exploring Network Structure, Dynamics, and Function using NetworkX,' in *Proc. 7th Python in Science Conf. (SciPy)*, pp. 11–15, 2008.

[14] M. Bostock, V. Ogievetsky, and J. Heer, 'D3: Data-Driven Documents,' *IEEE Trans. on Visualization and Computer Graphics*, vol. 17, no. 12, pp. 2301–2309, 2011.

[15] F. Pedregosa et al., 'Scikit-learn: Machine Learning in Python,' *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[16] P. Grinberg, 'Flask Web Development: Developing Web Applications with Python,' O'Reilly Media, 2nd ed., 2018.

[17] Sphoorthy Engineering College, 'Academic Regulations R23 for Autonomous B.Tech Programs,' SPHN Autonomous Academic Council, Hyderabad, 2023.

[18] National Assessment and Accreditation Council (NAAC), 'Institutional Accreditation Manual for Autonomous Colleges,' NAAC, Bengaluru, 2022.